Project 1: Asteroid Deflection by Instantaneous Change of Velocity

Andre Llaneta (wpl5304)

Spring 2026

Introduction

This project simulates an asteroid approaching Earth in a 2D under Newtonian gravity, and estimates the minimum instantaneous change in the asteroid's velocity (a "kick" Δv) needed to guarantee a miss (+ a chosen safety buffer). The asteroid begins far from Earth with the user prompted to input incoming speed, distance to apply kick and other necessary starting condtions. Its trajectory is found using Numerical Methods (my code uses Symplectic Euler). The kick is applied once, at the chosen kick radius r_k when the asteroid is inbound. My code then employs bracketing plus bisection (a common root finding technique) to find the minimum Δv that results in a miss.

The two test cases included in this report are:

- **Case 1 (Chixclub/Dinosaur asteroid):** very large mass, head-on approach, and an early kick radius (assuming hyperdrive/similar technology).
- **Case 2 (Mid sized city destroyer asteroid):** smaller mass, y axis starting offset (most asteroids dont come head on but get pulled in by Earth's gravity), and a kick radius chosen to be more feasible for interception.

Equations (Governing Model)

State variables and parameters

The simulation is 2D in an inertial frame with Earth fixed at the origin. The unknown state variables are the asteroid position and velocity:

$$\mathbf{r}(t) = \begin{bmatrix} x(t) \\ y(t) \end{bmatrix}, \qquad \mathbf{u}(t) = \dot{\mathbf{r}}(t) = \begin{bmatrix} u(t) \\ v(t) \end{bmatrix}.$$

The parameters the user inputs are:

$$m \text{ (asteroid mass)}, r_0 \text{ (initial distance)}, \quad v_0 \text{ (incoming speed)},$$
$$b \text{ (y axis starting offset)}, \quad h \text{ (safety buffer)}, \quad r_k \text{ (kick radius)}.$$

Two-body point-mass gravity

The governing equations of motion for the asteroid are

$$\dot{\mathbf{r}} = \mathbf{u}, \qquad \dot{\mathbf{u}} = -\frac{GM}{\|\mathbf{r}\|^3}\,\mathbf{r}.$$

In the code the components are written as,

$$\dot{x} = u, \qquad \dot{y} = v,$$
$$\dot{u} = -GM\frac{x}{(x^2+y^2)^{3/2}}, \qquad \dot{v} = -GM\frac{y}{(x^2+y^2)^{3/2}}.$$

With the scalar radius being

$$r(t) = \|\mathbf{r}(t)\| = \sqrt{x^2(t) + y^2(t)}.$$

Instantaneous change in velocity Δv (the "kick")

At the moment the asteroid reaches the chosen kick radius r_k, an instantaneous velocity change is applied:

$$\mathbf{u}^+ = \mathbf{u}^- + \Delta v\, \hat{\mathbf{n}},$$

where $\hat{\mathbf{n}}$ is chosen perpendicular to the current velocity direction, and its sign is chosen so the kick pushes the trajectory outward. In my code I form the unit normal

$$\hat{\mathbf{n}} = \begin{bmatrix} -v/s \\ u/s \end{bmatrix},$$

and compare it with the outward radial unit vector

$$\hat{\mathbf{r}} = \mathbf{r}/\|\mathbf{r}\| = \begin{bmatrix} x/r \\ y/r \end{bmatrix}.$$

If $\hat{\mathbf{n}} \cdot \hat{\mathbf{r}} > 0$, the kick uses $+\hat{\mathbf{n}}$; otherwise it uses $-\hat{\mathbf{n}}$, ensuring the kick is applied so that it changes the trajectory "outward".

Values to find

The key outcome is the minimum miss distance relative to Earth:

$$r_{\min} = \min_t r(t),$$

and the safety criterion is when the minimum radial distance from Earth is Earth's radius + the user chosen safety parameter

$$r_{\min} \geq R_E + h.$$

The code also evaluates the specific orbital energy (energy per unit asteroid mass)

$$\varepsilon(t) = \frac{1}{2}\|\mathbf{u}(t)\|^2 - \frac{GM}{r(t)}.$$

(Specific Orbital Energy Formula[1].)

When a kick is applied, the code computes the expected instantaneous specific-energy change from $\Delta\mathbf{u}$:

$$\Delta\varepsilon_{\text{expected}} = \mathbf{u}^- \cdot \Delta\mathbf{u} + \frac{1}{2}\|\Delta\mathbf{u}\|^2,$$

and compares it to the simulated change $\Delta\varepsilon_{\text{sim}} = \varepsilon^+ - \varepsilon^-$.
(Kinetic energy change identity [**?**].)

If the user chooses to use the rocket equation feature is enabled, the propellant mass is estimated via the ideal rocket equation:

$$\frac{m_2}{m_1} = \exp\left(-\frac{\Delta v}{U_{\text{frel}}}\right),$$

with U_{frel} being the effective exhaust velocity.
(Tsiolkovsky rocket equation [2].)

Numerical Method

Symplectic Euler

The trajectory equations are advanced in time using the symplectic Euler method, with timestep Δt:

$$u^{n+1} = u^n + \Delta t\, a_x(x^n, y^n), \qquad v^{n+1} = v^n + \Delta t\, a_y(x^n, y^n),$$

$$x^{n+1} = x^n + \Delta t\, u^{n+1}, \qquad y^{n+1} = y^n + \Delta t\, v^{n+1},$$

where the acceleration components are evaluated at time level n:

$$a_x(x^n, y^n) = -GM \frac{x^n}{((x^n)^2 + (y^n)^2)^{3/2}}, \qquad a_y(x^n, y^n) = -GM \frac{y^n}{((x^n)^2 + (y^n)^2)^{3/2}}.$$

Symplectic euler was chosen as its stable over long periods and avoids the energy drift of forward and backward Euler.
(Symplectic Euler properties[3].)

Kick detection

The kick is applied exactly once, when the simulated $r = r_k$. This is implemented by detecting a sign change in the inequality $r - r_k$ between successive timesteps:

$$(r^{n-1} > r_k) \;\wedge\; (r^n \le r_k)$$

where the radial velocity is computed as

$$v_r = \frac{\mathbf{r} \cdot \mathbf{u}}{\|\mathbf{r}\|}.$$

Minimum Δv search (bracketing + bisection)

Define the function

$$f(\Delta v) = r_{\min}(\Delta v) - (R_E + h).$$

The goal is to find the minimum Δv such that $f(\Delta v) \ge 0$. The code:

1. Starts with $\Delta v = 0$.
2. If $f(0) \ge 0$, returns $\Delta v_{\text{best}} = 0$ as it already misses even without a kick.
3. Otherwise, grows the upper bound Δv_{hi} (doubling each time) until $f(\Delta v_{\text{hi}}) \ge 0$ (Bracketing).
4. Uses bisection on $[\Delta v_{\text{lo}}, \Delta v_{\text{hi}}]$ to find the smallest Δv achieving $f(\Delta v) \ge 0$ within a relative tolerance:
   (a) Set $\Delta v_{\text{mid}} = \frac{1}{2}(\Delta v_{\text{lo}} + \Delta v_{\text{hi}})$.
   (b) Run the trajectory with Δv_{mid} and compute $f(\Delta v_{\text{mid}})$.
   (c) If $f(\Delta v_{\text{mid}}) \ge 0$, set $\Delta v_{\text{hi}} = \Delta v_{\text{mid}}$; otherwise set $\Delta v_{\text{lo}} = \Delta v_{\text{mid}}$.
   (d) Stop when the interval is sufficiently small; return Δv_{hi} as the minimum safe kick.

(Bisection method convergence [4].)

Validation

Kick energy consistency (validates logic of kick)

When a kick is applied, my code computes the expected instantaneous specific-energy change from $\Delta\mathbf{u}$. Since gravity is unchanged the $-GM/r$ terms cancels, so only kinetic energy changes:

$$\Delta\varepsilon = \frac{1}{2}\|\mathbf{u}^- + \Delta\mathbf{u}\|^2 - \frac{1}{2}\|\mathbf{u}^-\|^2.$$

Expanding the square gives

$$\Delta\varepsilon = \frac{1}{2}\Big(\|\mathbf{u}^-\|^2 + 2\,\mathbf{u}^- \cdot \Delta\mathbf{u} + \|\Delta\mathbf{u}\|^2\Big) - \frac{1}{2}\|\mathbf{u}^-\|^2 = \mathbf{u}^- \cdot \Delta\mathbf{u} + \frac{1}{2}\|\Delta\mathbf{u}\|^2.$$

My code uses this $\Delta\varepsilon_{\text{expected}}$ and compares it to the simulated change $\Delta\varepsilon_{\text{sim}} = \varepsilon^+ - \varepsilon^-$ as a consistency check.

Post-kick energy drift (checks accuracy of the timestep)

After the kick, the motion again follows two-body dynamics abd so the specific energy should remain approximately constant. Symplectic Euler does not conserve energy exactly, but for sufficiently small Δt the drift over the simulated time window should be small. The code reports the relative change between ε immediately after the kick and ε near the end of the simulation. If drift is large, it indicates the timestep is too large for the chosen parameters (this matters more for fast, close approaches).

Key Problems Encountered

U-turn path

Testing different parameters produced trajectories that visually resembled an unrealistic U-turn near Earth. This turned out to be primarily a parameter choice issue as when v_0 is too low, gravity strongly bends the path, and the asteroid can enter a highly curved trajectory. Increasing v_0 yielded the expected high-speed flyby geometry (slightly bends around earth). This was a good sanity check for the plots and to see if my simulation produced results that made sense qualitatively.

Angle of approach vs. y axis offset parameter intuition

In the beginning I decided to implement asteroid offset (when the asteroid doesn't approach earth directly head on) through varying the angle of approach. In this setup, the asteroid begins at $x = -r_0$ with $y = b$, an initial velocity pointing along $+x$ and an angle offset θ. However in practice, small differences in approach angle at distances like 10^{10} or 10^{11} m can translate into very large miss distances near Earth, which makes it hard to intuitively assume an approach angle.

Plot design: Earth view vs kick view

In the beginning I struggled to choose plots that showed the necessary information I wanted to convey:

- **Earth View** should zoom near closest approach to show whether the safety buffer is satisfied.
- **Kick View** should show the global geometry and the kick location relative to the overall inbound path.

The final plotting approach shows both the no kick and with kick trajectories, includes Earth and the safety buffer, and marks the closest approach points and kick point.

Results and Discussion

This section reports two input cases. Each case includes three figures:

1. a screenshot of the written results (console output)
2. Earth-view plot (zoom near Earth)
3. kick-view plot (global view)

Case 1: Chixclub Asteroid Simulation ("Dinosaur" Asteroid)

Inputs

- Asteroid mass: $m = 10 \times 10^{15}\,\mathrm{kg}$
- Initial speed: $v_0 = 20000\,\mathrm{m/s}$
- Y axis offset parameter: $b = 0\,\mathrm{m}$ (head on)

- Safety buffer: $h = 10^5\,\mathrm{m}$ (Earth's Atmospher)
- Kick radius: $r_k = 4 \times 10^{10}\,\mathrm{m}$ (early interception, could be achieved by nuclear powered systems, advanced plasma engines, photon based drives or other sci-fi esque solutions)
- Timestep: $\Delta t = 50\,\mathrm{s}$
- Rocket equation: Disabled

(Chixclub asteroid parameters [5].)

Written results

```
Asteroid mass m (kg) = 10e15
Initial speed (m/s) (don't choose anything <=0)= 20000
Y-axis starting offset parameter b (m) = 0
Safety buffer h (m) (Earth Athmosphere = 1e5) = 1e5
Kick radius rk (m) (must satisfy RE < rk < r0) = 4e10
Time step dt (s) [shoule be between 10 and 200 (lower -> accurate, higher -> faster)] = 50
Rocket equation? (1=yes,0=no) = 0

Baseline closest approach rmin = 5.455139e+06 m

Kick radius rk = 4.000000e+10 m
Required dv = 3.697998000 m/s
Energy = 6.837595e+16 J
Closest approach after kick rmin = 6.471002e+06 m
Post-kick specific-energy diagnostics:
  Simulated dE_kick = 6.837595e+00 J/kg
  Expected dE_kick  = 6.837595e+00 J/kg
  Kick dE rel error = 8.000295e-10 (tol = 1.0e-06)
  Kick energy check = PASS
  Post-kick drift   = 7.224908e-10 (tol = 1.0e-03)
  Drift check       = PASS
```

Figure 1: Case 1: Console output showing baseline $r_{\min}$, required Δv, energy, and validation tests

Earth view

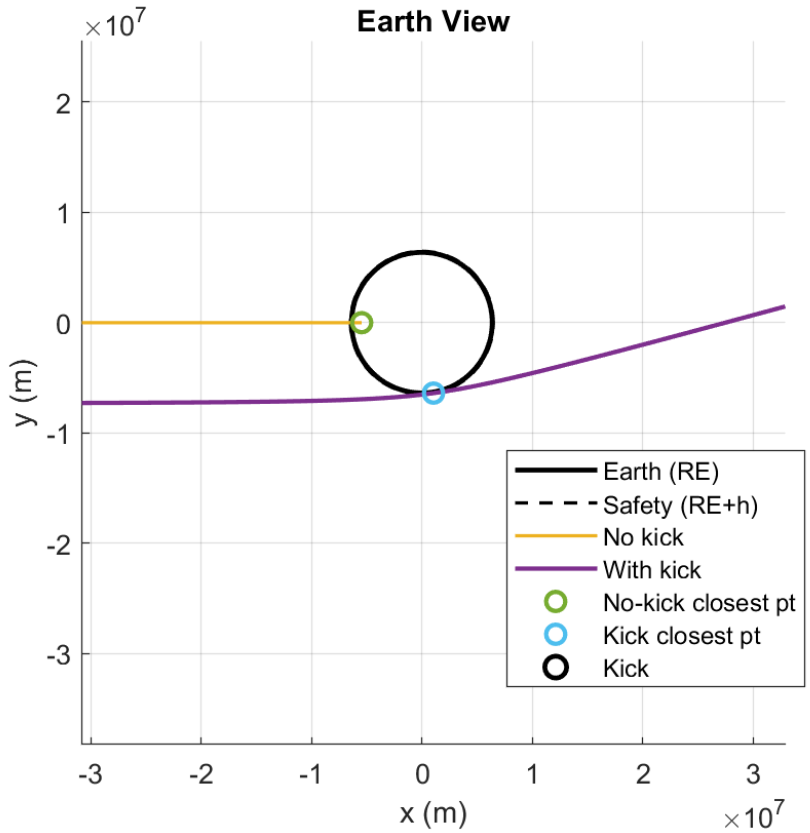

Figure 2: Case 1: Earth-view trajectory comparison (no kick vs with kick) with safety buffer circle (not really seen as Earths atmosphere << Earth's radius)

Kick view

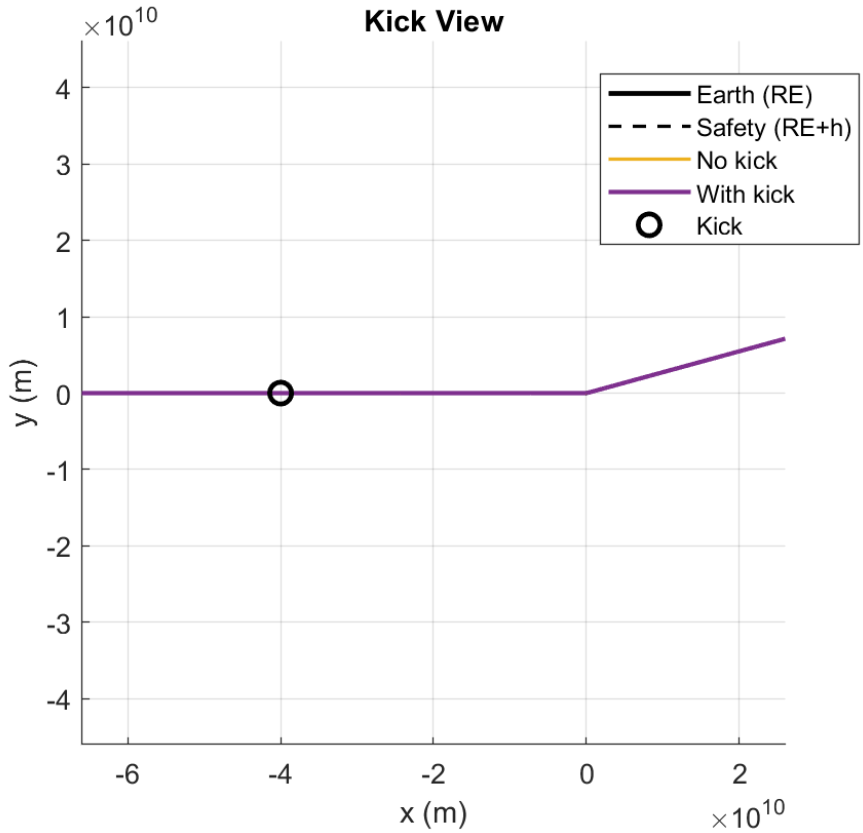

Figure 3: Case 1: Global trajectory comparison with marked kick location at $r = r_k$.

Discussion

This case questions what would have been required to prevent the asteroid that led to the extinction of the dinosaurs. With $b = 0$, the baseline trajectory is expected to collide without a Δv. The bisection search is therefore required to find Δv_{best} that produces $r_{\min} \geq R_E + h$. Because the kick is applied very early ($r_k = 4 \times 10^{10}$ m), it only requires $\Delta v = 3.698$ m/s as the small change accumulates into a meaningful miss distance by the time the asteroid reaches Earth. The calculated impulse energy is $E_b = 6.837595 \times 10^{16}$ J, which is about 1.09×10^3 Hiroshima bombs (one bomb is about 6.276×10^{13} J). Validations of energy shows that the simulation behaves at set tolerances.

Case 2: Mid-sized asteroid with a more feasible intercept

Inputs

- Asteroid mass: $m = 1.26 \times 10^{10}$ kg
- Initial speed: $v_0 = 2000$ m/s
- Y axis offset parameter: $b = 5 \times 10^6$ m
- Safety buffer: $h = 10^5$ m
- Timestep: $\Delta t = 50$ s
- Rocket equation: Enabled

- Exhaust velocity: $U_{\text{frel}} = 3000\,\text{m/s}$
  (typical chemical engine value)[6]

Choosing a feasible kick radius

I chose r_k using a time-to-impact estimate. The asteroid approaches at a uniform at speed v_0. If an intercept must occur at least $T_{\text{detection}}$ seconds before closest approach, then a reasonable kick radius is

$$r_k \approx v_0\, T_{\text{detection}}.$$

For a more realistic interception I take a detection time of about one month:

$$T_{\text{lead}} = 30 \text{ days} = 30 \times 24 \times 3600 \text{ s} = 2.592 \times 10^6 \text{ s}.$$

Then

$$r_k \approx (2000 \text{ m/s})(2.592 \times 10^6 \text{ s}) = 5.184 \times 10^9 \text{ m} \approx 5 \times 10^9 \text{ m}.$$

Thus I set

$$r_k = 5 \times 10^9\,\text{m},$$

which corresponds to about 30 days of lead time at $v_0 = 2$ km/s .

Written results

```
Asteroid mass m (kg) = 1.26e10
Initial speed (m/s) (don't choose anything <=0)= 2000
Y-axis starting offset parameter b (m) = 5e6
Safety buffer h (m) (Earth Athmosphere = 1e5) = 1e5
Kick radius rk (m) (must satisfy RE < rk < r0) = 5e9
Time step dt (s) [shoule be between 10 and 200 (lower -> accurate, higher -> faster)] = 50
Rocket equation? (1=yes,0=no) = 1
U_frel (m/s) (Typically 2000-4500 for standard chemical engines) = 3000

Baseline closest approach rmin = 6.353841e+06 m

Kick radius rk = 5.000000e+09 m
Required dv = 12.592240000 m/s
Energy = 9.989564e+11 J
Closest approach after kick rmin = 6.471003e+06 m
Post-kick specific-energy diagnostics:
  Simulated dE_kick = 7.928225e+01 J/kg
  Expected dE_kick  = 7.928225e+01 J/kg
  Kick dE rel error = 1.344329e-12 (tol = 1.0e-06)
  Kick energy check = PASS
  Post-kick drift   = 4.078689e-07 (tol = 1.0e-03)
  Drift check       = PASS
Propellant mass = 5.299856e+07 kg
m(t2)/m(t1) = 9.958114e-01
```

Figure 4: Case 2: Console output showing baseline $r_{\min}$, required Δv, energy, and validation

Earth view

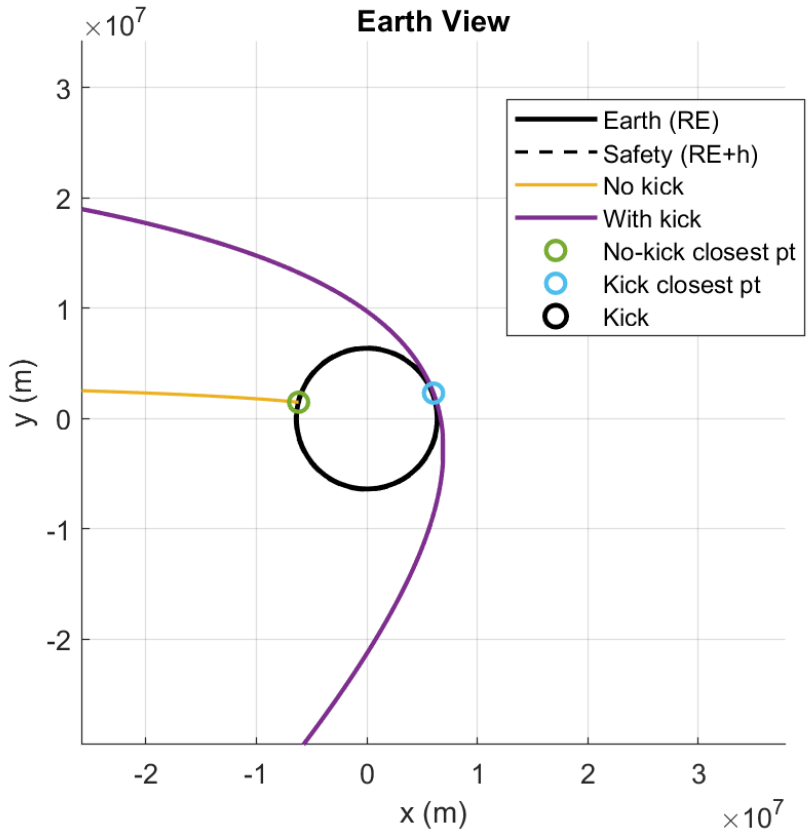

Figure 5: Case 2: Earth-view trajectory comparison (no kick vs with kick) with safety buffer circle (not really seen as Earths atmosphere << Earth's radius)

Kick view

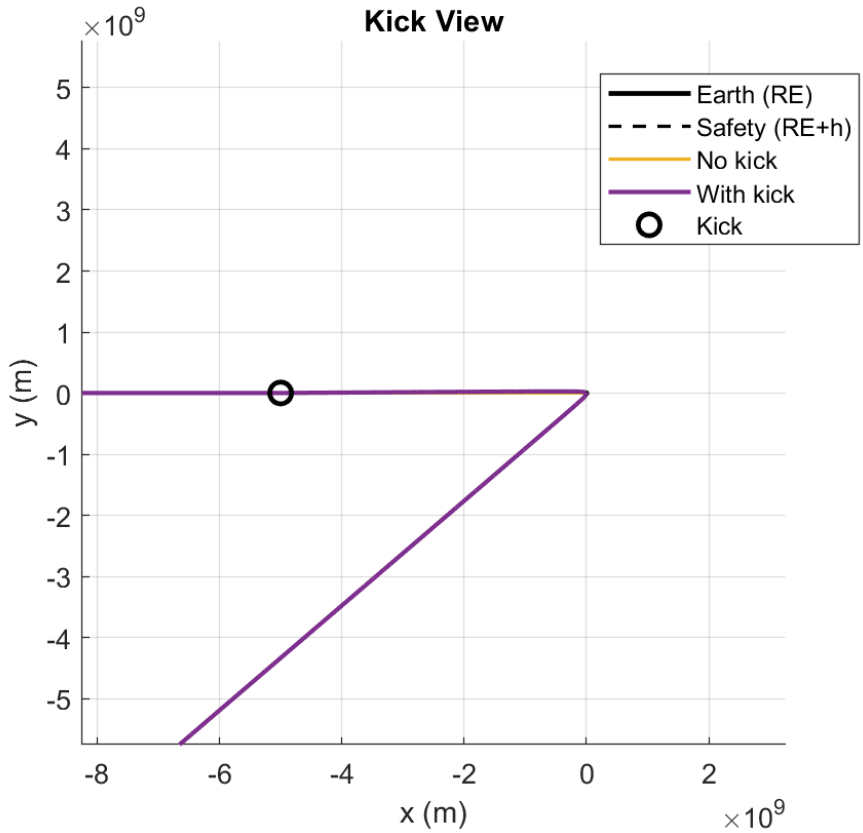

Figure 6: Case 2: Global trajectory comparison with marked kick location at $r = r_k$

Discussion

This case questions what it takes to deflect a mid sized city destroyer asteroid at a more feasible kick radius. With $b = 5 \times 10^6$ m, the simulation finds that the asteroid is still expected to collide without a Δv. The bisection search is therefore required to find Δv_{best} that produces $r_{\min} \geq R_E + h$. The kick is closer to earth at a distance more feasible to reach with current rechnology ($r_k = 5 \times 10^9$ m), it requires a large $\Delta v = 12.592$ m/s as there is less distance for the change to accumulate by the time the asteroid reaches Earth. The calculated impulse energy is much less at $E_b = 9.989 \times 10^{11}$ J, which is about 0.16 one Hiroshima bomb (one bomb is about 6.276×10^{13} J). This lower energy required is due to a mix of a much slower approach speed and a lower asteroid mass. Validations of energy shows that the simulation behaves at set tolerances. In this case I set the rocket equation on and it found that it would require about 5.299×10^7 kg of rocket fuel which is about 11.52 Starship + Super Heavy rockets. This is computed by assuming a fully fueled Starship + Super Heavy carries about 4.6×10^6 kg of propellant, so

$$N = \frac{m_{\text{prop}}}{m_{\text{stack}}} = \frac{5.299 \times 10^7}{4.6 \times 10^6} \approx 11.52.$$

Conclusions

This project implemented a 2D asteroid approach model with an instantaneous velocity kick and used bracketing plus bisection to compute the minimum Δv required to guarantee a miss distance of at least $R_E + h$. The governing equations were the Newtonian two-body equations, and carried forward using symplectic Euler. Validation relied on (i) matching the analytically expected energy change across the kick and (ii) monitoring post kick energy drift.

References

[1] J. Peraire, *Lecture D29: Central Force Motion: Orbits* (MIT OpenCourseWare / MIT DSpace). Visible link: https://dspace.mit.edu/bitstream/handle/1721.1/60691/16-07-fall-2004/contents/lecture-notes/d29.pdf

[2] NASA Glenn Research Center, *Ideal Rocket Equation*. Visible link: https://www1.grc.nasa.gov/beginners-guide-to-aeronautics/ideal-rocket-equation/

[3] M. R. Roussel, *Symplectic integration methods* (University of Lethbridge lecture slides, 2019). Visible link: https://people.uleth.ca/~roussel/C4010nonlin/slides/12symplectic.pdf

[4] *Numerical Analysis, 9th ed.*, Section 2.1 "The Bisection Method" (PDF). Visible link: https://faculty.ksu.edu.sa/sites/default/files/numerical_analysis_9th.pdf

[5] H. J. Durand-Manterola and G. Cordero-Tercero, *Assessments of the energy, mass and size of the Chicxulub Impactor*, arXiv:1403.6391 (2014). Visible link: https://arxiv.org/abs/1403.6391

[6] L. P. Forsley, *High Isp Propulsion* (NASA Glenn Research Center PDF). Includes the statement that chemical rockets have $v_e < 4.4 \times 10^3$ m/s. Visible link: https://www1.grc.nasa.gov/wp-content/uploads/High-Isp-Propulsion-LCF.pdf

Appendix: Full Code Listing

```
% CSCI-UA 330 Project 1
% Asteroid deflection with symplectic Euler + bisection for minimum dv
% Andre Llaneta - wpl5304

clear; clc; close all

% constants
G = 6.67430e-11; % gravitational constant (m^3/kg/s^2)
M = 5.97219e24; % mass of Earth (kg)
RE = 6.371e6; % radius of Earth (m)

% inputs
m_ast = input("Asteroid mass m (kg) = ");
v0 = input("Initial speed (m/s) (don't choose anything <=0)= ");
b = input("Y-axis starting offset parameter b (m) = ");
h = input("Safety buffer h (m) (Earth Athmosphere = 1e5) = ");
rk = input("Kick radius rk (m) (must satisfy RE < rk < r0) = ");
dt = input("Time step dt (s) [shoule be between 10 and 200 (lower -> accurate, higher -> faster)] =
     ");
useRocket = input("Rocket equation? (1=yes,0=no) = ");

if useRocket==1
    Ufrel = input("U_frel (m/s) (Typically 2000-4500 for standard chemical engines) = "); %
        velocity of expelled propellant (higher -> more efficient, but more expensive) For chemical
         rockets, typically between 2000 and 4500 m/s
else
    Ufrel = NaN;
end

% setup and initial conditions
r0 = 1e11; % start distance (m) (far enough to be effectively at infinity, but not too far to cause
     numerical issues)
Rstop = RE; % impact stop radius (stops when it hits the Earth)
Rsafe = RE + h; % miss criterion (earth + a safety buffer)
Rdone = r0; % stop when outbound and far from Earth (for better looking plot)

if ~(rk > Rstop && rk < r0)
    % rk must be between Earth and the initial radius so the kick is applied on the modeled inbound
         trajectory
    error("Choose a different rk: must satisfy RE < rk < r0. Your r0 is %g m", r0)
end

tmax = 20*r0/max(v0,1); % max time (s) (20 chosen as safety factor to allow for some orbits, but
     not too long to cause numerical issues)
clockmax = ceil(tmax/dt); % rounds up as tmax might not be an exact multiple of dt

% start with baseline of no deflection, then increase dv by bisection until it just misses (rmin >=
     Rsafe)
dv = 0; % change of velocity at rk (m/s)

% run baseline to get rmin without deflection, and to store the trajectory for plotting
[xsave0, ysave0, rmin0, xK0, yK0] = runtraj(dv, G, M, r0, b, v0, rk, dt, clockmax, Rstop, Rdone);

fprintf("\nBaseline closest approach rmin = %.6e m\n", rmin0)

% if already misses, no need to deflect
```

```
if rmin0 >= Rsafe
    fprintf("Already misses (>= RE+h). Required dv = 0\n");
    dv_best = 0;
    Eb = 0;
    mprop = 0;
    xsave = xsave0; ysave = ysave0;
    xK = NaN; yK = NaN;
    rmin_def = rmin0;
else
    % bracket dv so f(dv)=rmin(dv)-Rsafe crosses 0
    dv_lo = 0;
    dv_hi = 1e-6; % starts small to avoid overshooting, grows fast so it converges quickly

    for i = 1:50 % max 50 iterations to find upper bound but should converge much faster than that
        [~, ~, rmin_hi] = runtraj(dv_hi, G, M, r0, b, v0, rk, dt, clockmax, Rstop, Rdone);
        f_hi = rmin_hi - Rsafe;

        if f_hi >= 0 % found an upper bound that misses
            break
        end

        dv_hi = 2*dv_hi; % doubles dv_hi if it still hits

        if dv_hi > 1e6 % avoids infinite loop if dt too large or rk too small
            error("Couldn't bracket dv. Try smaller dt, bigger b, or earlier rk.")
        end
    end

    [~, ~, rmin_hi] = runtraj(dv_hi, G, M, r0, b, v0, rk, dt, clockmax, Rstop, Rdone);
    f_hi = rmin_hi - Rsafe;

    % checks if we found an upper bound that misses after 50 iterations
    if f_hi < 0
        error("Couldn't bracket dv. Try smaller dt, bigger b, or earlier rk.")
    end

    % bisection to find minimum dv that achieves f(dv)=rmin(dv)-Rsafe >= 0
    for i = 1:50
        dv_mid = 0.5*(dv_lo + dv_hi); % finds midpoint dv between current bounds

        [~, ~, rmin_mid] = runtraj(dv_mid, G, M, r0, b, v0, rk, dt, clockmax, Rstop, Rdone);
        f_mid = rmin_mid - Rsafe;

        % changes upper or lower bound depending on if dv_mid achieves a miss or not
        if f_mid >= 0
            dv_hi = dv_mid;
        else
            dv_lo = dv_mid;
        end

        % checks if the bounds are close enough to stop
        if (dv_hi - dv_lo) <= 1e-6*dv_hi
            break
        end
    end

    dv_best = dv_hi;

    % final deflected run to store trajectory + kick point + energy diagnostics
```

```
[xsave, ysave, rmin_def, xK, yK, dE_kick_expected, dE_kick_sim, eps_post, eps_end] = runtraj(
    dv_best, G, M, r0, b, v0, rk, dt, clockmax, Rstop, Rdone);

% finds kinetic energy of the deflection in J
Eb = 0.5*m_ast*dv_best^2;

if useRocket==1
    % rocket equation: m(t2)/m(t1) = exp(-||U2-U1||/||U_frel||)
    % ||U2-U1|| = dv_best, and m(t2)=m_ast
    m2 = m_ast;
    m1 = m2*exp(dv_best/Ufrel);
    mprop = m1 - m2;
    mr = m2/m1;
else
    mprop = NaN;
    mr = NaN;
end

% print results
fprintf("\nKick radius rk = %.6e m\n", rk)
fprintf("Required dv = %.9f m/s\n", dv_best)
fprintf("Energy = %.6e J\n", Eb)
fprintf("Closest approach after kick rmin = %.6e m\n", rmin_def)

% verify kick energy change and post-kick energy drift
rel_tol_kick = 1e-6; % tolerance for kick energy error (should be very small since it's not
    numerical)
rel_tol_drift = 1e-3; % tolerance for post-kick energy drift (larger since it's numerical)
rel_kick_err = abs(dE_kick_sim - dE_kick_expected)/max(1e-12, abs(dE_kick_expected)); %
    compares simulated and expected specific energy change from the kick
rel_drift = abs(eps_end - eps_post)/max(1e-12, abs(eps_post)); % compares specific orbital
    energy after kick and at ends of simulation to check for energy drift due to numerical
    errors

fprintf("Post-kick specific-energy diagnostics:\n")
fprintf(" Simulated dE_kick = %.6e J/kg\n", dE_kick_sim)
fprintf(" Expected dE_kick = %.6e J/kg\n", dE_kick_expected)
fprintf(" Kick dE rel error = %.6e (tol = %.1e)\n", rel_kick_err, rel_tol_kick)

% checks if the kick energy error is within tolerance
if rel_kick_err <= rel_tol_kick
    fprintf(" Kick energy check = PASS\n")
else
    fprintf(" Kick energy check = FAIL\n")
end

% checks if the post kick energy drift is within tolerance
fprintf(" Post-kick drift = %.6e (tol = %.1e)\n", rel_drift, rel_tol_drift)
if rel_drift <= rel_tol_drift
    fprintf(" Drift check = PASS\n")
else
    fprintf(" Drift check = FAIL (try smaller dt)\n")
end

% print rocket equation results
if useRocket==1
    fprintf("Propellant mass = %.6e kg\n", mprop)
    fprintf("m(t2)/m(t1) = %.6e\n", mr)
end
```

```
end

% plots
th = linspace(0,2*pi,400);
xe = RE*cos(th); ye = RE*sin(th);
xs = Rsafe*cos(th); ys = Rsafe*sin(th);

% Earth view
% radii along each saved trajectory
r0_traj = hypot(xsave0, ysave0);
r1_traj = hypot(xsave, ysave);

% indices of closest approach
[~, i0] = min(r0_traj); % no kick closest approach index
[~, i1] = min(r1_traj); % with kick closest approach index

% center on deflected closest approach
xc = xsave(i1);
yc = ysave(i1);

figure; hold on; grid on; axis equal

% draw Earth + safety buffer
plot(xe, ye, 'k-', 'LineWidth', 1.8)
plot(xs, ys, 'k--', 'LineWidth', 1.2)

% plot full trajectories
plot(xsave0, ysave0, 'LineWidth', 1.2)
plot(xsave, ysave, 'LineWidth', 1.6)

% mark closest-approach points
plot(xsave0(i0), ysave0(i0), 'o', 'MarkerSize', 7, 'LineWidth', 1.6)
plot(xsave(i1), ysave(i1), 'o', 'MarkerSize', 7, 'LineWidth', 1.6)

% mark kick if present
if ~isnan(xK)
    plot(xK, yK, 'ko', 'MarkerSize', 8, 'LineWidth', 1.8)
end

title("Earth View")
xlabel("x (m)"); ylabel("y (m)")

zoomRE = 5; % zoom in around Earth to show closest approach

xlim(xc + zoomRE*RE*[-1 1]);
ylim(yc + zoomRE*RE*[-1 1]);

legend("Earth (RE)", "Safety (RE+h)", "No kick", "With kick", ...
       "No-kick closest pt", "Kick closest pt", "Kick", ...
       'Location','best')

% Kick view
figure; hold on; grid on; axis equal

% draw Earth + safety buffer
plot(xe, ye, 'k-', 'LineWidth', 1.8)
plot(xs, ys, 'k--', 'LineWidth', 1.2)

% plot full trajectories
```

```
plot(xsave0, ysave0, 'LineWidth', 1.2)
plot(xsave, ysave, 'LineWidth', 1.6)

% mark kick if present
if ~isnan(xK)
    plot(xK, yK, 'ko', 'MarkerSize', 8, 'LineWidth', 1.8)
end
title("Kick View")
xlabel("x (m)"); ylabel("y (m)")
if ~isnan(xK)
    c1 = 0.5*xK; % centers the plot at half the kick radius
    c2 = 0.5*yK;
    span = 1.15*max(sqrt(xK^2+yK^2), 50*RE); % sets the span to be slightly larger than the kick
        radius, but at least 50 RE so it's not too zoomed in
    xlim(c1 + span*[-1 1]); % sets x limits centered at c1 with the calculated span
    ylim(c2 + span*[-1 1]); % sets y limits centered at c2 with the calculated span
else
    zoomKick = 3000; % if no kick, just zoom in around the Earth to show the trajectories
    xlim(zoomKick*RE*[-1 1]);
    ylim(zoomKick*RE*[-1 1]);
end
legend("Earth (RE)", "Safety (RE+h)", "No kick", "With kick", "Kick", 'Location', 'best')



function [xsave, ysave, rmin, xK, yK, dE_kick_expected, dE_kick_sim, eps_post, eps_end] = runtraj(
    dv, G, M, r0, b, v0, rk, dt, clockmax, Rstop, Rdone)

    % initial conditions
    x = -r0; % assumes asteroid starts on the left and moves to the right
    y = b; % Y-axis starting offset parameter
    u = v0; % initial velocity
    v = 0; % no initial vertical velocity, just horizontal

    % creates an array of zeroes to store the trajectory for plotting
    xsave = zeros(1,clockmax);
    ysave = zeros(1,clockmax);

    rmin = 1e99; % starts high so any approach will be smaller, updated every step to find closest
        approach

    % initializes variables to store the kick
    kicked = 0;
    xK = NaN; yK = NaN;
    dE_kick_expected = NaN;
    dE_kick_sim = NaN;
    eps_post = NaN;
    eps_end = NaN;

    turned = 0; % becomes 1 after it misses
    r_prev = sqrt(x^2 + y^2); % stores the previous radius

    for clock = 1:clockmax

        % finds current radius and updates minimum radius
        r = sqrt(x^2 + y^2);
        if r < rmin
            rmin = r;
        end
```

```
% stop if impact
if r <= Rstop
    xsave = xsave(1:clock-1);
    ysave = ysave(1:clock-1);
    eps_end = 0.5*(u^2 + v^2) - G*M/max(r, 1e-12);
    return
end

% radial velocity
vr = (x*u + y*v)/max(r,1e-12);
if vr > 0 % distance from earth increasing meaning asteroid is outbound
    turned = 1;
end

% stop if done
if turned==1 && r >= Rdone
    xsave = xsave(1:clock-1);
    ysave = ysave(1:clock-1);
    eps_end = 0.5*(u^2 + v^2) - G*M/max(r, 1e-12);
    return
end

% kick trigger: crosses rk (only once)
if kicked==0 && (r_prev > rk) && (r <= rk)
    u_pre = u;
    v_pre = v;
    eps_pre = 0.5*(u_pre^2 + v_pre^2) - G*M/max(r, 1e-12); % specific orbital energy
        immediately before the kick
    s = sqrt(u^2 + v^2); % speed magnitude
    nx = -v/s; % unit normal vector perpendicular to velocity
    ny = u/s; % points to the left of the velocity vector

    rx = x/r; % unit radial vector from Earth to asteroid
    ry = y/r; % points from Earth to asteroid

    % applies the kick in the direction that increases the radius
    if (nx*rx + ny*ry) > 0 % dot product between normal and radial unit vector
        u = u + dv*nx; % updates velocity with the kick
        v = v + dv*ny; % updates velocity with the kick
        du = dv*nx; % change in velocity components due to the kick
        dvv = dv*ny; % used for energy change calculation
    else
        u = u - dv*nx;
        v = v - dv*ny;
        du = -dv*nx;
        dvv = -dv*ny;
    end

    eps_post = 0.5*(u^2 + v^2) - G*M/max(r, 1e-12); % specific orbital energy immediately
        after the kick
    dE_kick_sim = eps_post - eps_pre; % simulated specific energy change from the kick
    dE_kick_expected = u_pre*du + v_pre*dvv + 0.5*(du^2 + dvv^2); % expected specific energy
        change from the kick
    kicked = 1;
    xK = x;
    yK = y;
end
```

```
        r_prev = r;

        % symplectic Euler
        u = u - dt*G*M*x/(r^3); % updates velocity with gravitational acceleration
        v = v - dt*G*M*y/(r^3);

        x = x + dt*u;
        y = y + dt*v;

        xsave(clock) = x;
        ysave(clock) = y;
    end

    % specific orbital energy at end of simulation
    r_end = sqrt(x^2 + y^2);
    eps_end = 0.5*(u^2 + v^2) - G*M/max(r_end, 1e-12);
end
```